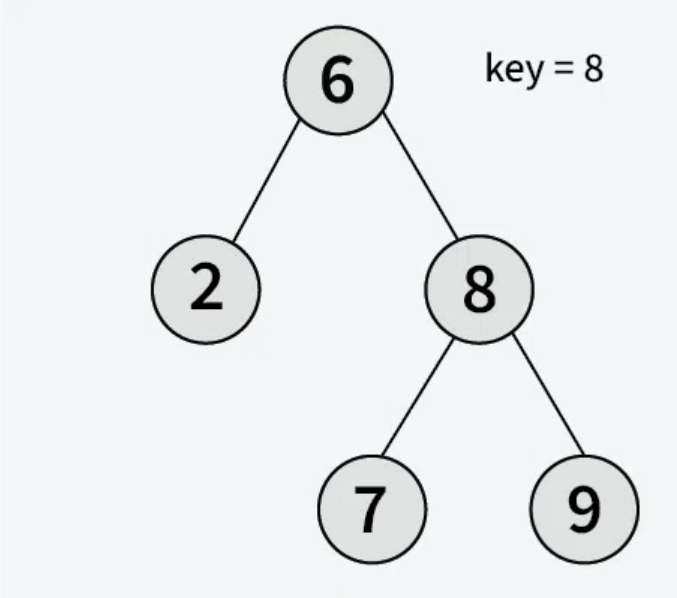
Searching in Binary Search Tree (BST)

Last Updated : 25 Sep, 2024

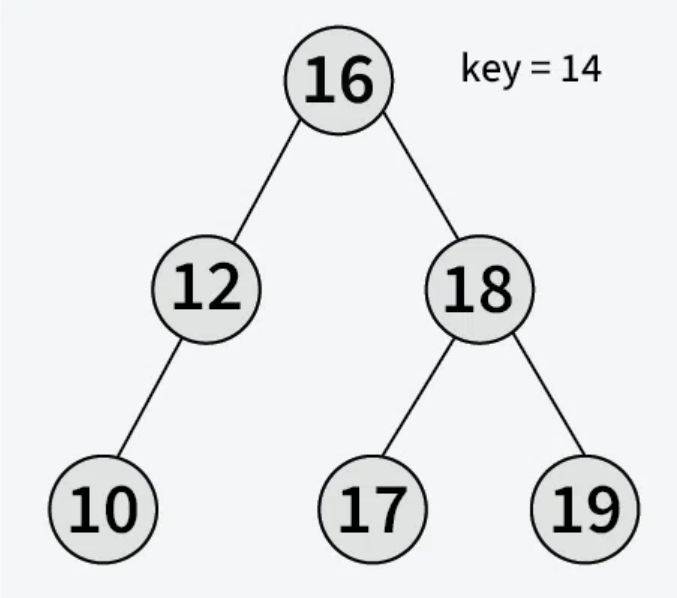
Given a **BST**, the task is to search a node in this **BST**. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm.

Input: *Root of the below BST*



Output: *True*
Explanation: *8 is present in the BST as right child of root*

Input: *Root of the below BST*



Output: *False*
Explanation: *14 is not present in the BST*

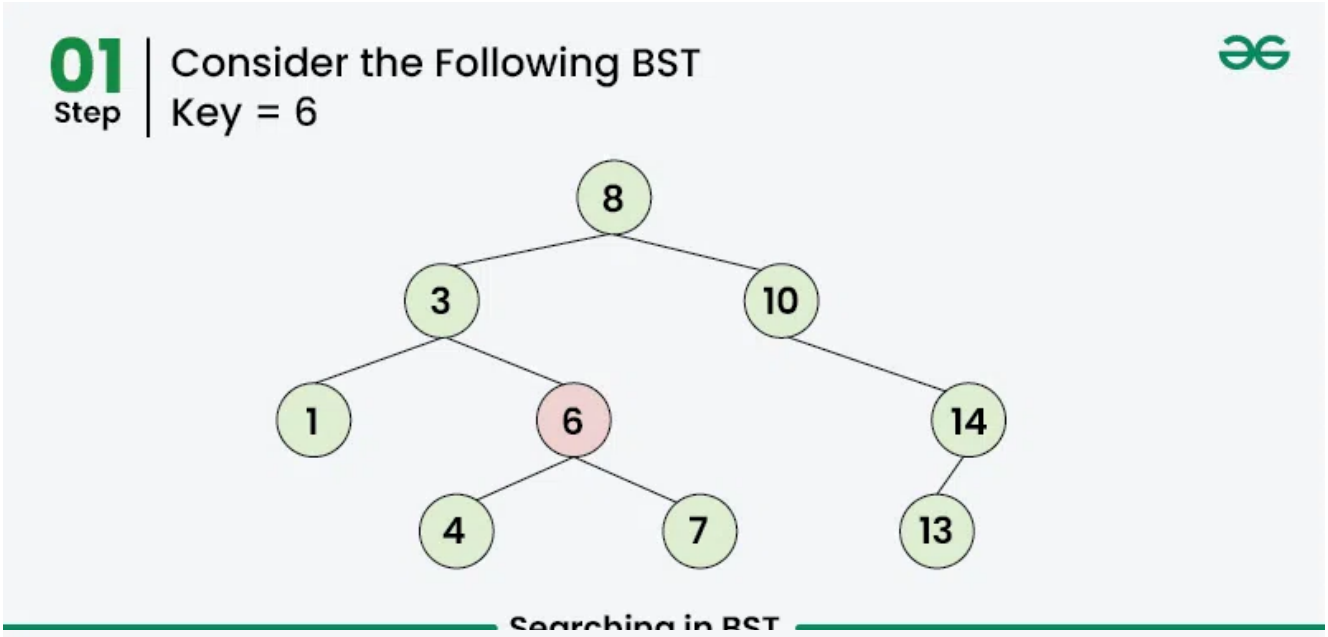
Algorithm to search for a key in a given Binary Search Tree:

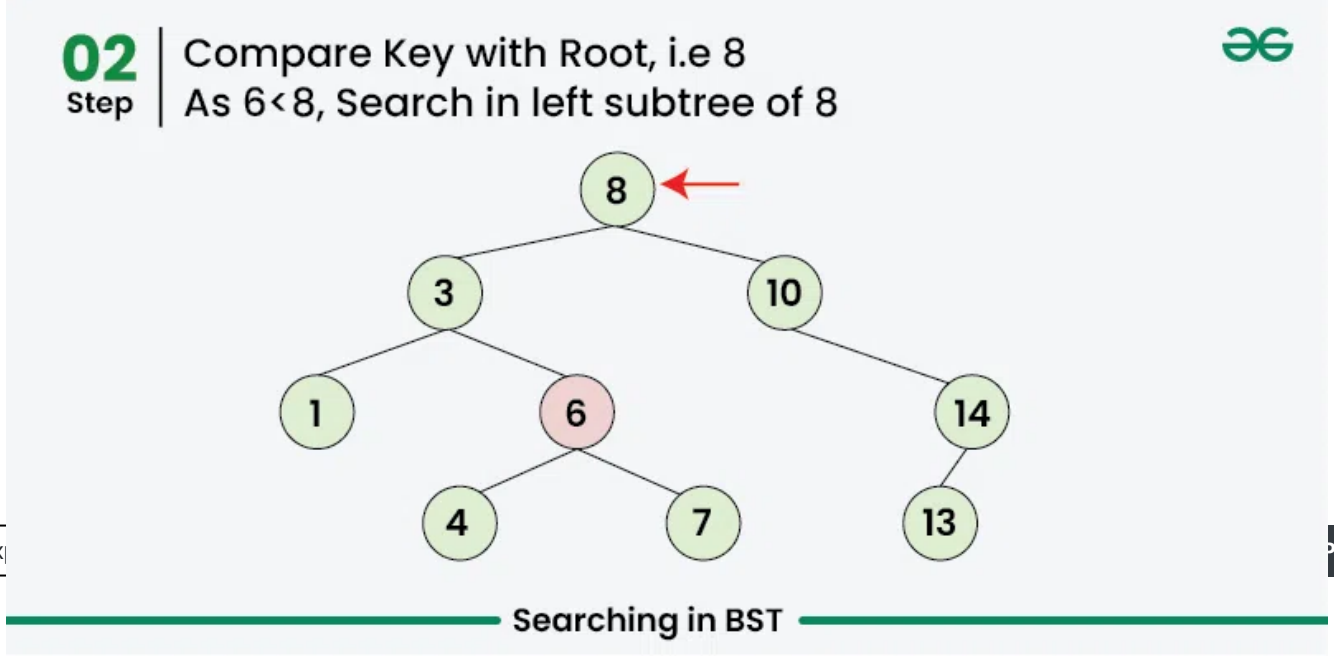
Let's say we want to search for the number **X,** We start at the root. Then:

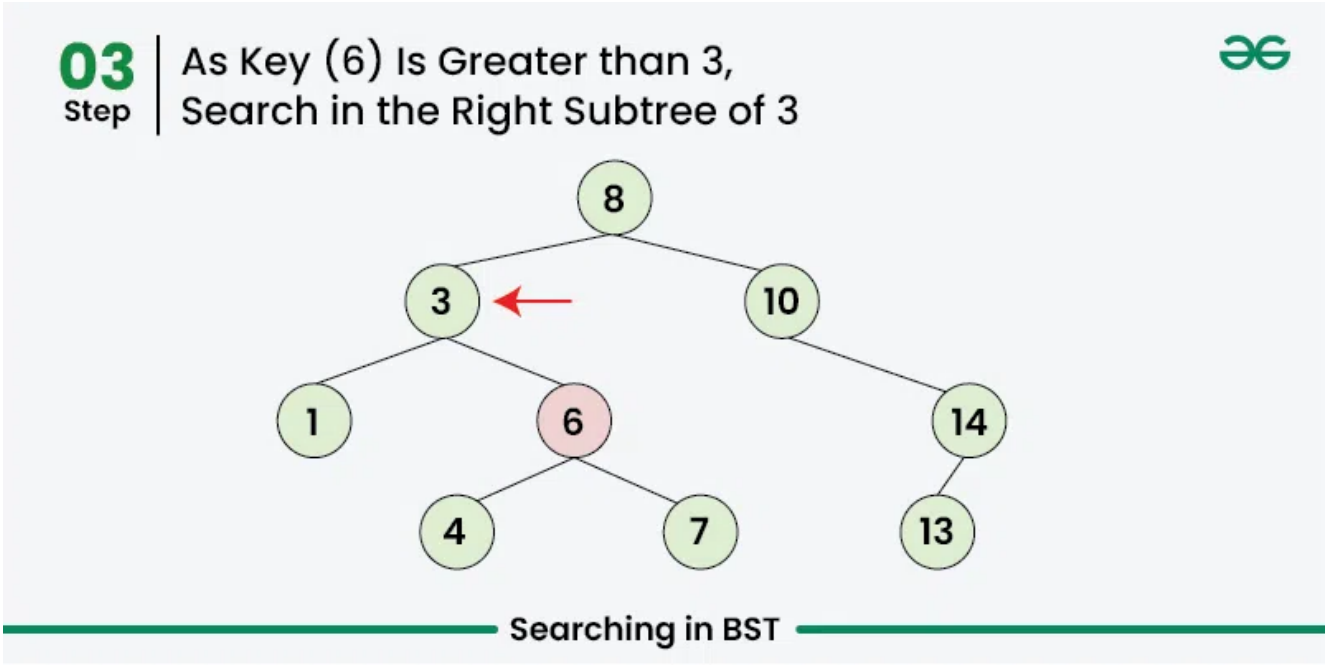
- We compare the value to be searched with the value of the root.
    - If it's equal we are done with the search if it's smaller we know that we need to go to the left subtree because in a binary search tree all the elements in the left subtree are smaller and all the elements in the right subtree are larger.
- Repeat the above step till no more traversal is possible
- If at any iteration, key is found, return True. Else False.

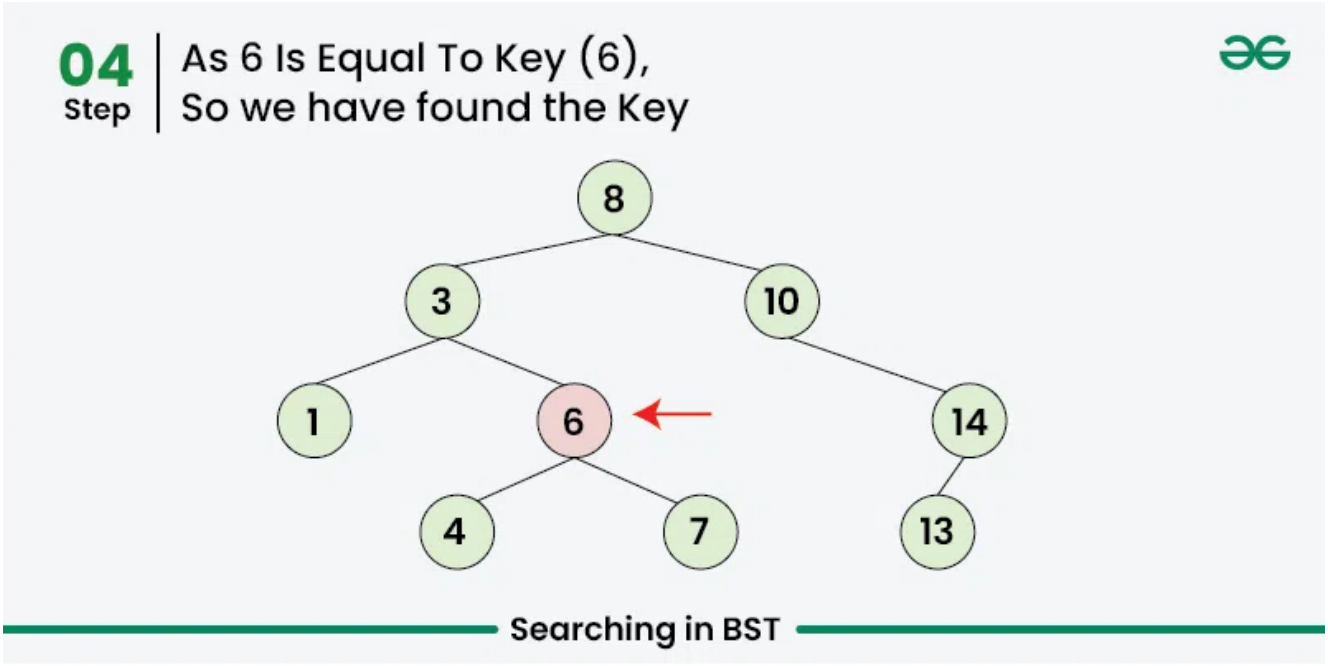
Illustration of searching in a BST:

See the illustration below for a better understanding:









Recursive Program to implement search in BST:

C++ C Java Python C# JavaScript

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int item) {
        key = item;
        left = right = NULL;
    }
};

// function to search a key in a BST
Node* search(Node* root, int key) {

    // Base Cases: root is null or key
    // is present at root
    if (root == NULL || root->key == key)
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}

// Driver Code
int main() {

    // Creating a hard coded tree for keeping
    // the length of the code small. We need
    // to make sure that BST properties are
    // maintained if we try some other cases.
    Node* root = new Node(50);
    root->left = new Node(30);
    root->right = new Node(70);
    root->left->left = new Node(20);
    root->left->right = new Node(40);
    root->right->left = new Node(60);
    root->right->right = new Node(80);

    (search(root, 19) != NULL)? cout << "Found\n":
```

```
                                cout << "Not Found\n";


    (search(root, 80) != NULL)? cout << "Found\n":
                                cout << "Not Found\n";

    return 0;
}
```

Output

```
Not Found
Found
```

Time complexity: O(h), where h is the height of the BST.
Auxiliary Space: O(h) This is because of the space needed to store the recursion stack.

We can avoid the auxiliary space and recursion overhead withe the help of iterative implementation. Below is link for the iterative implementation that works in O(h) time and O(1) auxiliary space.

Iterative Program to implement search in BST

Next Article
Deletion in Binary Search Tree (BST)

Similar Reads

Binary Search Tree

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right...

3 min read

Introduction to Binary Search Tree

Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its left subtree...

3 min read

Applications of BST

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

2 min read

Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST.Example: How to Insert a value in a Binary Search Tree:A new key is always inserted at the leaf by maintaining the property of the binary search...

15 min read

Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

7 min read

Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:Case 1. Delete a Leaf Node in BSTDeletion in BSTCase 2. Delete a Node with Single Child in BSTDeleting a...

10 min read

Binary Search Tree (BST) Traversals â€“ Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree.Â Input:Â A Binary Search TreeOutput:Â Inorder Traversal: 10 20 30 100 150...

10 min read

Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height.Examples: Input: Output: Explanation: The above unbalanced BST...

10 min read

Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The heigh...

4 min read